Breaking and Fixing Spoed

Yan Jia[1,2], Peng Wang[3], Lei Hu[1,2], Gang Liu[4],
Tingting Guo[5], and Shuping Mao[6]

[1] State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[2] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
[3] School of Cryptology, University of Chinese Academy of Sciences, Beijing, China
[4] National Key Laboratory of Security Communication, Chengdu, China
[5] School of Cyber Science and Engineering, Ningbo University of Technology, Zhejiang, China
[6] Beijing Electronic Science & Technology Institute, Beijing, China

Abstract. Spoed is an authenticated encryption scheme based on compression functions. We show that Spoed fails to achieve its claimed security guarantees with respect to both integrity and confidentiality. In particular, we present a universal forgery attack that succeeds with probability one using only a single encryption query and a single decryption query. The attack exploits a structural weakness in the feedback mechanism of Spoed, allowing internal inputs of the underlying pseudorandom function to coincide during verification. We further show that the same weakness enables efficient plaintext-recovery attacks, permitting recovery of almost the entire plaintext with at most two decryption queries, depending on the associated-data length. We explain why the original security proof of Spoed fails, and identify the mismatch between the collision events used in the H-coefficient analysis and the schemes actual behavior. Finally, we propose a minimally modified variant, fSpoed, and prove that it achieves the originally claimed security bounds under standard assumptions.

Keywords: Authenticated Encryption · Spoed

1 Introduction

Authenticated encryption (AE) schemes built from cryptographic compression functions form a distinct design line that departs from traditional block-cipher-based modes. By leveraging well-studied hash-function primitives, this line aims to enable efficient processing of associated data while retaining strong provable security guarantees. A representative construction in this family is OMD, which was introduced in the CAESAR competition [CMN+15,CMN+14a,CMN+14b]. Subsequent works proposed several variants to improve efficiency and simplify security analysis [RVV14,RVV15,AM16].

Among these designs, the scheme Spoed, proposed by Ashur and Mennink [AM16], was introduced as a simplification of the earlier pure-OMD (p-OMD) construction in the nonce-respecting setting, where nonces are guaranteed

to be distinct across encryptions. While p-OMD achieves almost-free handling of associated data, its structure involves a large number of case distinctions depending on the lengths of the message and the associated data, resulting in a complex scheme description and a lengthy security proof.

The central design goal of Spoed is to eliminate this structural complexity while retaining the efficiency and security guarantees of p-OMD. This goal is achieved through a generalized padding mechanism, called GPAD, which injectively encodes both the associated data and the message into blocks of size $2n$. As a consequence, Spoed admits a single, uniform processing rule for all inputs, significantly reducing pre-computation overhead and yielding a substantially shorter and more transparent security proof. Indeed, the authors emphasize that the proof of Spoed is about one quarter the size of that of p-OMD, and that the resulting simplification leads to improved security bounds and reduces the risk of errors hidden in complex case analyses.

Our contributions. Despite these advantages, we show that the claimed security of Spoed does not hold. In this paper, we present a series of attacks that fundamentally break the integrity and confidentiality guarantees of Spoed. Most notably, we describe a universal forgery attack that succeeds with probability one using only a single encryption query and a single decryption query. These queries involve only short messages and ciphertexts of comparable length, and the attack requires neither heavy pre-computation nor expensive computation: a single bit operation suffices to carry it out. Importantly, the attack is independent of the underlying compression function h used in the construction of Spoed, including its input and output lengths.

We extend our techniques to the plaintext-recovery setting. At the cost of a small increase in the number of encryption and decryption queries, an adversary can recover almost the entire plaintext, depending on the length of the associated data. Taken together, our results demonstrate that the vulnerabilities of Spoed are structural in nature and comparable in severity to classic breaks of authenticated encryption schemes, such as the attacks on OCB2 [IIMP19,IIMP20].

Beyond demonstrating the insecurity of Spoed, we further investigate the root cause of the attacks. We show that the failure of Spoed is not due to the choice of the underlying compression function, but rather stems from a structural weakness in its feedback mechanism, which allows internal states to coincide during verification.

Motivated by this observation, we propose a minimally modified variant, called fSpoed, which alters only a single component of the original construction. Despite its simplicity, this modification suffices to eliminate the identified vulnerabilities while preserving the overall design philosophy and efficiency of Spoed. We prove that fSpoed achieves the originally claimed confidentiality and integrity guarantees in the nonce-respecting setting under standard assumptions.

2 Preliminaries

Notations. We write n for the state size. Let $\{0,1\}^*$ denote the set of all finite binary strings, including the empty string ε. For a string X, let $|X|$ be its bit length, and let $\langle X \rangle_b$ denote the b-bit encoding of $|X|$. We use $\oplus$ for the bitwise exclusive-or operation. Both $\otimes$ and $\cdot$ denote multiplication in the finite field $\mathbf{GF}(2^n)$. An element $a_{n-1} \dots a_1 a_0 \in \mathbf{GF}(2^n)$ is interpreted either as the integer $A = \sum_{i=0}^{n-1} 2^i a_i$ or equivalently as the polynomial $a(x) = a_{n-1}x^{n-1} + \dots + a_1 x + a_0$. We identify the field elements 2 and 3 with the polynomials x and $x+1$, respectively, in $\mathbf{GF}(2^n)$. For a bit string X, let $\mathrm{left}_n(X)$ denote the leftmost n bits of X. By $X_1 \| \cdots \| X_a \xleftarrow{n} X$, we denote the decomposition of X into consecutive n-bit blocks, where $|X_1| = \cdots = |X_{a-1}| = n$ and $0 < |X_a| \leq n$. The leftmost bit is considered the most significant bit. For a finite set A, the notation $a \xleftarrow{\$} A$ denotes uniform sampling of an element a from A.

Authenticated Encryption. An authenticated encryption scheme Π consists of two algorithms: an encryption algorithm $\mathcal{E}$ and a decryption algorithm $\mathcal{D}$. The encryption algorithm

$$\mathcal{E} : (K, N, A, M) \to (C, T)$$

takes as input a secret key K, a nonce N, associated data (AD) A, and a message M, and outputs a ciphertext C together with an authentication tag T. Throughout this paper, we consider tag-based authenticated encryption schemes, which is consistent with the specification of Spoed.

The decryption algorithm

$$\mathcal{D} : (K, N, A, C, T) \to M \,/\, \perp$$

outputs either the plaintext M if the input tuple is valid, or the special symbol $\perp$ indicating failure otherwise.

Let \$ denote a random function that, on any fresh input tuple (N, A, M), outputs a uniformly random pair $\{0,1\}^{|M|} \times \{0,1\}^\tau$.

Security Model. An adversary $\mathcal{A}$ is a probabilistic algorithm that is given oracle access to a set of oracles $\mathcal{O}$, and is denoted by $\mathcal{A}^{\mathcal{O}}$. We write $1 \leftarrow \mathcal{A}^{\mathcal{O}}$ to indicate that $\mathcal{A}$ outputs 1.

In our setting, the adversary $\mathcal{A}$ is given access to either the real encryption oracle $\mathcal{E}_K$ or its ideal counterpart \$, and may additionally have access to the decryption oracle $\mathcal{D}_K$, where the secret key $K \leftarrow \{0,1\}^k$ is sampled uniformly at random during the initialization of each game.

We say that $\mathcal{A}$ is *nonce-respecting* (nr) if it never issues two encryption queries that use the same nonce. Equivalently, all encryption queries made by $\mathcal{A}$ are required to have distinct nonce values.

Definition 1 (AE Security). *Let $\Pi = (\mathcal{E}, \mathcal{D})$ be an authenticated encryption scheme. For any adversary $\mathcal{A}$, we define its advantage in breaking the confidentiality of Π as*

$$\mathbf{Adv}^{\mathrm{conf}}_{\Pi}(\mathcal{A}) = \left| \Pr\left[K \xleftarrow{\$} \{0,1\}^k,\, 1 \leftarrow \mathcal{A}^{\mathcal{E}_K}\right] - \Pr\left[1 \leftarrow \mathcal{A}^{\$}\right] \right|.$$

Procedure: $\mathrm{GPAD}_{n,\tau}$
Input: A, X
// A is AD and X is either a message or a ciphertext.
Output: Z
// Z has length of $\ell = \max(m, \lceil \frac{a+m}{2} \rceil) + 1$ where a and m are the number of n-bits blocks of A and X, respectively.
$A_1 \| \ldots \| A_a \xleftarrow{n} A$
$X_1 \| \ldots \| X_m \xleftarrow{n} X$
if $a \leq m$ **then**
  $Z_1 \leftarrow \langle \tau \rangle_n \| A_1$
  for $i = 2$ **to** a **do**
    $Z_i = X_{i-1} \| A_i$
  for $i = a+1$ **to** m **do**
    $Z_i \leftarrow X_{i-1} \| 0^n$
  $Z_l \leftarrow X_m \| \mathrm{len}\,(A, X)$
else
  $Z_1 \leftarrow \langle \tau \rangle_n \| A_1$
  for $i = 2$ **to** $m+1$ **do**
    $Z_i = X_{i-1} \| A_i$
  for $i = m+2$ **to** $\lceil \frac{a+m}{2} \rceil$ **do**
    $Z_i \leftarrow A_{2(i-1)-m} \| A_{2(i-1)-m+1}$
  if $a + m \bmod 2 = 0$ **then** $Z_l \leftarrow A_a \| \mathrm{len}\,(A, X)$
  else $Z_l \leftarrow 0^n \| \mathrm{len}\,(A, X)$
return $Z_1, \ldots, Z_l$

Fig. 1. The GPAD padding function

Let $\mathbf{Adv}^{\mathrm{conf}}_{\Pi}(\mathsf{nr}, q, \ell, \sigma, t)$ *denote the maximum of* $\mathbf{Adv}^{\mathrm{conf}}_{\Pi}(\mathcal{A})$ *taken over all nonce-respecting adversaries* $\mathcal{A}$ *that run in time at most* t*, make at most* q *encryption queries, process a total of at most* σ *message blocks, and whose longest query consists of at most* ℓ *blocks.*

For any adversary $\mathcal{A}$ *that never queries the decryption oracle on a ciphertext returned by the encryption oracle, we define its advantage in breaking the integrity of* Π *as*

$$\mathbf{Adv}^{\mathrm{int}}_{\Pi}(\mathcal{A}) = \Pr\left[K \xleftarrow{\$} \{0,1\}^k,\ \mathcal{A}^{\mathcal{E}_K, \mathcal{D}_K} \text{ forges}\right].$$

Let $\mathbf{Adv}^{\mathrm{int}}_{\Pi}(\mathsf{nr}, q_{\mathcal{E}}, q_{\mathcal{D}}, \ell, \sigma, t)$ *denote the maximum of* $\mathbf{Adv}^{\mathrm{int}}_{\Pi}(\mathcal{A})$ *taken over all nonce-respecting adversaries* $\mathcal{A}$ *that run in time at most* t*, make at most* $q_{\mathcal{E}}$ *encryption queries and* $q_{\mathcal{D}}$ *decryption queries, process a total of at most* σ *blocks, and whose longest query consists of at most* ℓ *blocks.*

Generalized Padding Scheme. Spoed employs a unified padding mechanism, called the generalized padding function, denoted by

$$\mathrm{GPAD}_{n,\tau} : \{0,1\}^* \times \{0,1\}^* \to \left(\{0,1\}^{2n}\right)^+.$$

The purpose of $\mathrm{GPAD}_{n,\tau}(A, M)$ is to encode the associated data A and the message M into a single sequence of fixed-length blocks, so that the subsequent

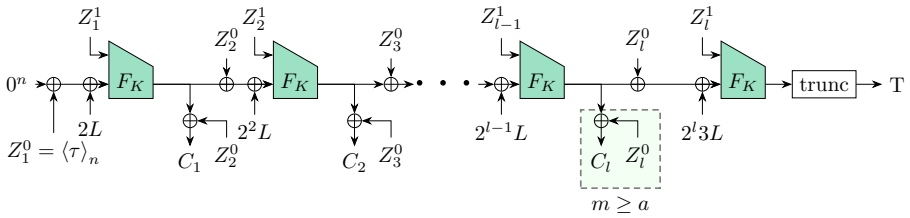


Fig. 2. Spoed, where $L = F_K(N\|0)$ and $Z_i = Z_i^0\|Z_i^1$, $i = 1, \cdots, l$.

encryption and verification procedures can be described using a uniform processing rule, independent of the relative lengths of the inputs.

Given two arbitrary but finite strings, namely the associated data A and a message M (or a ciphertext in the decryption procedure), GPAD outputs a sequence of $2n$-bit blocks $Z_1, \cdots, Z_l$ via an injective mapping. Each output block consists of two n-bit halves ($Z_i = Z_i^0\|Z_i^1$, $i = 1, \cdots, l$), which are constructed by interleaving blocks of the message (or ciphertext) with blocks of the associated data according to their lengths. To ensure unambiguous decoding and length binding, GPAD appends a dedicated length-encoding field to the final block, defined as

$$\mathrm{len}(A, M) = \langle|A|\rangle_{n/2} \,\|\, \langle|M|\rangle_{n/2}.$$

More specifically, we restate the padding rules of GPAD in functional form in Fig. 1.

Spoed *and its functions.* Spoed is illustrated in Fig. 2 and the pseudo code is give in Fig. 4. Let h denote an underlying compression function that takes a $2n$-bit input and an n-bit chaining value, and outputs an n-bit chaining value. Based on h, Spoed instantiates a keyed compression function, denoted by F_K, which is obtained by prefixing the secret key K to the input of h. Concretely, F_K is evaluated by placing K in the leftmost n bits of the input to h.

PRF and tweak PRF. Let $\mathrm{Func}\,(2n, n)$ be the set of all functions that map $2n$-bit inputs to n-bit outputs. For any adversary $\mathcal{A}$, we define its advantage in breaking PRF security of F by

$$\mathbf{Adv}_{\mathrm{F}}^{\mathrm{prf}}(\mathcal{A}) = \left| \begin{array}{l} \Pr\left[K \xleftarrow{\$} \{0,1\}^k ; 1 \leftarrow \mathcal{A}^{F_K}\right] \\ -\Pr\left[f \xleftarrow{\$} \mathrm{Func}\,(2n, n) ; 1 \leftarrow \mathcal{A}^{f}\right] \end{array} \right|.$$

Similarly, let $\widetilde{\mathrm{Func}}\,(\mathcal{T}, 2n, n)$ be the set of all functions that take an additional input tweak $T \in \mathcal{T}$ and map tweaks and $2n$-bit inputs to n-bit outputs. For any adversary $\mathcal{A}$, we define its advantage in breaking the $\widetilde{\mathrm{tPRF}}$ security of $\widetilde{\mathrm{F}}$ by

$$\mathbf{Adv}_{\widetilde{\mathrm{F}}}^{\widetilde{\mathrm{tprf}}}(\mathcal{A}) = \left| \begin{array}{l} \Pr\left[K \xleftarrow{\$} \{0,1\}^k ; 1 \leftarrow \mathcal{A}^{\widetilde{F}_K}\right] \\ -\Pr\left[\widetilde{f} \xleftarrow{\$} \widetilde{\mathrm{Func}}\,(\mathcal{T}, 2n, n) ; 1 \leftarrow \mathcal{A}^{f}\right] \end{array} \right|.$$

3 Structural Weaknesses of Spoed

3.1 Security Claims of Spoed

In [AM16], Ashur and Mennink established the following security bounds for Spoed.

Theorem 1 ([AM16]). *We have*

$$\mathbf{Adv}^{\mathrm{conf}}_{\mathrm{Spoed}}(\mathsf{nr}, q, \ell, \sigma, t) \leq \frac{1.5\,\sigma^2}{2^n} + \mathbf{Adv}^{\mathrm{prf}}_{F}(2\sigma, t'),$$

$$\mathbf{Adv}^{\mathrm{int}}_{\mathrm{Spoed}}(\mathsf{nr}, q_{\mathcal{E}}, q_{\mathcal{D}}, \ell, \sigma, t) \leq \frac{1.5\,\sigma^2}{2^n} + \frac{\ell q_{\mathcal{D}}}{2^n} + \frac{q_{\mathcal{D}}}{2^\tau} + \mathbf{Adv}^{\mathrm{prf}}_{F}(2\sigma, t'),$$

where $t' \approx t$.

3.2 Probability-1 Universal Forgery

We first illustrate our forgery attack on a minimal non-trivial instance, and then extend it to general message and associated-data lengths. The forgery attack exploits the fact that Spoed does not enforce sufficient linkage between consecutive blocks during verification, allowing an adversary to construct a fresh decryption query whose final compression-function input coincides with that of a previous encryption query.

A minimal example. Consider the extreme case where the nonce, the associated data, and the message all consist of a single n-bit block, i.e.,

$$|N| = |A| = |M| = n.$$

In this setting, the generalized padding function produces exactly two blocks, denoted by Z_1 and Z_2, and thus $\ell = 2$.

According to the specification of Spoed given in Fig. 4, the GPAD encoding of (A, M) is

$$Z_1 = \langle \tau \rangle_n \,\|\, A, \qquad Z_2 = M \,\|\, \langle n \rangle_{n/2} \,\|\, \langle n \rangle_{n/2}.$$

Moreover, the masking value is computed as

$$L = F_K(N \,\|\, 0^n).$$

Forgery strategy. A nonce-respecting adversary $\mathcal{A}$ proceeds as follows, as illustrated in Fig. 3.

(i) Choose an arbitrary nonce $N \in \{0,1\}^n$ and arbitrary values $A, A', M \in \{0,1\}^n$ such that $A \neq A'$.
(ii) Query the encryption oracle on (N, A, M) and obtain a ciphertext-tag pair (C, T).
(iii) Make a *fresh* decryption query (N, A', C, T).

The goal of the adversary is solely to make the decryption query pass verification; the returned plaintext, if any, is irrelevant for the forgery attack.

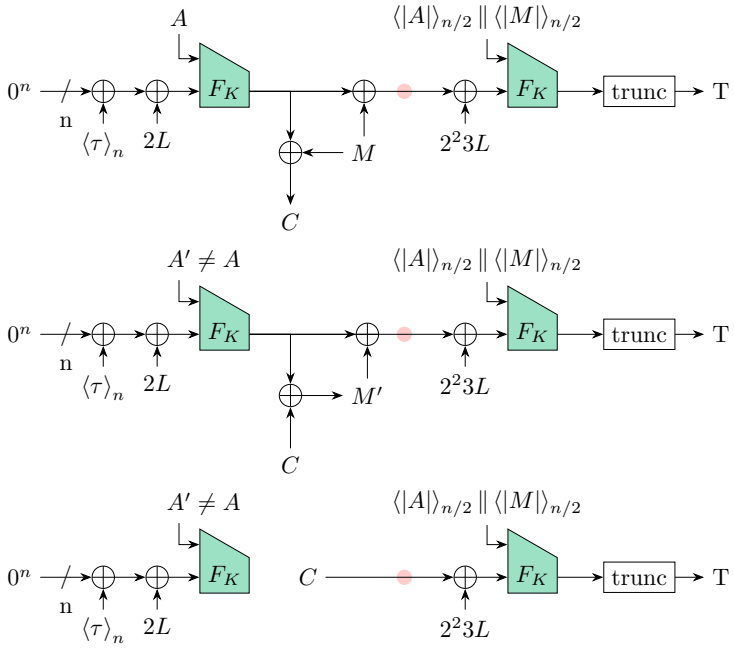


Fig. 3. Illustration of our forgery attack. **Top:** encryption query to $\mathcal{E}$. **Middle:** decryption query to $\mathcal{D}$. **Bottom:** an equivalent illustration of the middle one.

Why verification succeeds. In both the original encryption query and the forgery attempt, the input to the second evaluation of the keyed compression function F_K coincides. Indeed, in both cases, the left half of the second GPAD block satisfies $Z_2^0 = C$, and hence the internal state immediately before the final F_K-evaluation equals

$$C \oplus 2^2 3L.$$

Consequently, the final evaluation is performed on the identical input

$$t_2 = F_K\big((C \oplus 2^2 3L) \,\|\, \langle n\rangle_{n/2} \,\|\, \langle n\rangle_{n/2}\big)\,,$$

which yields the same output in both executions. Since the authentication tag T is derived by truncating t_2, the verification in the decryption query necessarily succeeds.

Therefore, a single encryption query followed by a single, fresh decryption query suffices to produce a valid forgery. In particular, for some small constant t, we have

$$\mathbf{Adv}^{\mathrm{int}}_{\mathrm{Spoed}}(\mathsf{nr}, 1, 1, 2, 4, t) = 1.$$

Extension to general lengths. The above attack extends naturally to arbitrary lengths of the associated data and the message. The key observation is that

as long as the final input to the last F_K-evaluation during verification coincides with that of a previous encryption query, the authentication tag will be accepted.

Lemma 1. *Let (C, T) be a valid ciphertext-tag pair returned by the* Spoed *encryption algorithm under (N, A, M). Let C_i and A_i denote the i-th n-bit blocks of C and A, respectively. For any tuple (N, A', C', T), where C'_i and A'_i denote the i-th n-bit blocks of C' and A', if*

$$\begin{aligned} C'_m &= C_m, \\ A'_j &= A_j \quad \textit{for all } j = m+1, \ldots, a, \\ \mathrm{len}(C') &= \mathrm{len}(C), \\ \mathrm{len}(A') &= \mathrm{len}(A), \end{aligned}$$

then the decryption query (N, A', C', T) passes the Spoed *verification with probability* 1.

Note that when $a \leq m$, the second condition above degenerates.

Proof. Let $\ell = \max\left(m, \lceil \frac{a+m}{2} \rceil\right) + 1$. According to the specification given in Appendix A [AM16], we have two cases:

$$\begin{aligned} t_i &= F_K(t_{i-1} \oplus 2^i L \oplus Z_i^0 \| Z_i^1) \text{ for all } \ell > i \geq m+1 \quad \text{and} \\ t_\ell &= F_K(t_{\ell-1} \oplus 2^\ell 3L \oplus Z_\ell^0 \| Z_i^1). \end{aligned}$$

As $Z'^0_{m+1} = C'_m = C_m = Z^0_{m+1} = t_{i-1} \oplus M_{i-1}$ holds, two internal states collide, i.e., $t'_{m+1} = t_{m+1}$. When the relation $a > m+1$ holds, all following Z_i and Z'_i are the same, it leads to a final state collision $t'_\ell = t_\ell$. On the other hand, the relation $a \leq m+1$ holds, the C_m itself is the last block Z_{m+1}, which also leads the final state collision because have the same input $C_m \oplus 2^\ell 3L \| Z^1_\ell$ on calling F_K.

3.3 Plaintext Recovery Attack

We now show that the probability-1 forgery attack from Section 3.2 can be further leveraged to recover the plaintext. The key observation is that once an adversary is able to construct fresh decryption queries that always pass verification, the decryption oracle can be used as a controllable probe to extract information about the internal keystream and, consequently, the plaintext.

Throughout this section, we assume that the adversary is given a valid ciphertext–tag pair (N, A, C, T) returned by the encryption oracle. Let $m = \lceil |C|/n \rceil$ and $a = \lceil |A|/n \rceil$ denote the numbers of n-bit blocks of the ciphertext and the associated data, respectively.

General strategy. By Lemma 1, as long as the final input to the last evaluation of the keyed compression function F_K remains unchanged, a decryption query will pass verification with probability 1, regardless of modifications made to earlier blocks. This allows the adversary to modify selected blocks of the associated data or the ciphertext while preserving the validity of the tag, and to observe the resulting plaintext returned by the decryption oracle.

We distinguish three cases according to the length of the associated data.

Case 1: $|A| > n$ *and* $m > 1$. In this case, the adversary can recover the full plaintext using two decryption queries.

Specifically, let $A^{(1)}$ (resp., $A^{(2)}$) be an associated-data string that differs from A only in its first (resp., second) n-bit block. By Lemma 1, both decryption queries $(N, A^{(1)}, C, T)$ and $(N, A^{(2)}, C, T)$ pass verification and return corresponding plaintexts $M^{(1)}$ and $M^{(2)}$.

In the first query, all plaintext blocks except the first one coincide with the original plaintext M; in the second query, the first plaintext block can be recovered while the remaining blocks except the second one remain unchanged. Combining the outputs of these two queries allows the adversary to reconstruct the entire plaintext M.

Case 2: $0 < |A| \leq n$. In this case, the adversary can recover all plaintext blocks except possibly the first one using a single decryption query.

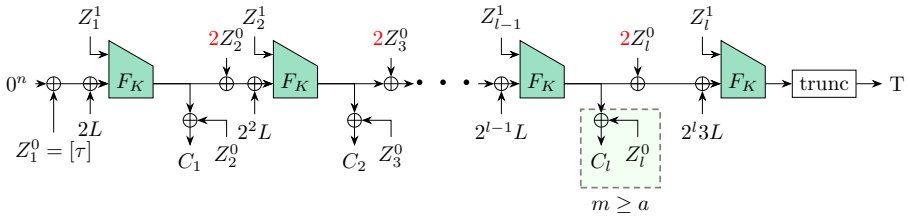
Let A' be an associated-data string that differs from A only in its first n-bit block. By Lemma 1, the decryption query (N, A', C, T) passes verification and returns a plaintext M'. All blocks of M' except the first one coincide with the original plaintext M, allowing the adversary to recover $M_2, \ldots, M_m$.

If $m > 1$, the adversary can additionally recover the first plaintext block. To this end, construct a ciphertext C' that differs from C only in its first n-bit block, and query the decryption oracle on (N, A, C', T). This query also passes verification. Let the returned plaintext block be denoted by M'_1. Since, by the specification of Spoed, the first plaintext block satisfies $M_1 = C_1 \oplus t_1$, the adversary can derive the internal value t_1 from C'_1 and M'_1 through $t_1 = C'_1 \oplus M'_1$, and hence recover the original plaintext block M_1.

Case 3: $|A| = 0$. When the associated data is empty, the adversary can recover all plaintext blocks except the last one, provided that $m > 1$.

In this setting, the adversary modifies the second-to-last ciphertext block while keeping the last block and the length encoding unchanged. By Lemma 1, the resulting decryption query passes verification and returns a plaintext that coincides with the original plaintext on the first $m - 2$ blocks. Using the same keystream-recovery technique as in Case 2, the adversary can also recover the $(m-1)$-th plaintext block. The final plaintext block, however, cannot be recovered without altering the final input to the last F_K-evaluation and thus invalidating the tag.

Discussion. The above cases show that the plaintext-recovery capability of the adversary depends on the length of the associated data. In all cases where $m > 1$, the adversary can recover either the full plaintext or all but one plaintext block. This limitation stems from the fact that the final plaintext block is tightly coupled with the input used to generate the authentication tag, and any attempt to extract additional information would necessarily invalidate the verification procedure.



Encryption $\mathcal{E}$

Input : (K, N, A, M)
Output: (C, T)

1: $(Z_1, \ldots, Z_l) = \mathrm{GPAD}_{n,\tau}(A, M)$
2: $m = \lceil |M| / n \rceil$
3: $L = F_K(N \| 0)$
4: $t_0 = 0^n$
5: **for** $i = 1$ **to** $\ell - 1$ **do**
6: $\quad t_i = F_K\left(t_{i-1} \oplus 2^i L \oplus 2Z_i^0 \| Z_i^1\right)$
7: $\quad C_i = t_i \oplus Z_{i+1}^0$
8: **end**
9: $t_\ell = F_K\left(t_{l-1} \oplus 2^\ell 3L \oplus 2Z_\ell^0 \| Z_\ell^1\right)$
10: $C = \mathrm{left}_{|M|}(C_1 \| \cdots \| C_{\ell-1})$
11: $T = \mathrm{left}_\tau(t_\ell)$
12: **return** (C, T)

Decryption $\mathcal{D}$

Input : (K, N, A, C, T)
Output: $M/\perp$

1: $(Z_1, \ldots, Z_l) = \mathrm{GPAD}_{n,\tau}(A, C)$
2: $m = \lceil |C| / n \rceil$
3: $\rho = |C| \bmod n$
4: $L = F_K(N \| 0)$
5: $t_0 = 0^n, M_0 = Z_1^0$
6: **for** $i = 1$ **to** $\ell - 1$ **do**
7: $\quad t_i = F_K\left(t_{i-1} \oplus 2^i L \oplus 2M_{i-1} \| Z_i^1\right)$
8: $\quad$ **if** $i < m$ **then** $M_i = t_i \oplus Z_{i+1}^0$
9: $\quad$ **if** $i = m$ **then** $M_i = \mathrm{left}_\rho(t_i) \oplus Z_{i+1}^0$
10: $\quad$ **if** $i > m$ **then** $M_i = Z_{i+1}^0$
11: **end**
12: $t_\ell = F_K\left(t_{l-1} \oplus 2^\ell 3L \oplus 2M_{\ell-1} \| Z_\ell^1\right)$
13: $M = \mathrm{left}_{|C|}(M_1 \| \cdots \| M_{\ell-1})$
14: $T' = \mathrm{left}_\tau(t_\ell)$
15: **return** $T = T' ? M : \perp$

Fig. 4. Spoed and its fixed variant fSpoed. The highlighted red parts introduce a fixed public constant (namely, the constant 2), while all other components of the construction remain identical to those of Spoed. In the decryption line 12, the original term Z_i^0 appears to be a typographical error and should be $M_{\ell-1}$, see Appendix A.

4 A Minimal Repair: The fSpoed Construction

fSpoed. We propose a fixed variant of Spoed, denoted by fSpoed. The construction of fSpoed differs from Spoed in only a single statement in the state update, as highlighted in Fig. 4. Despite its minimal nature, this modification fundamentally changes how differences propagate across successive evaluations of the keyed compression function.

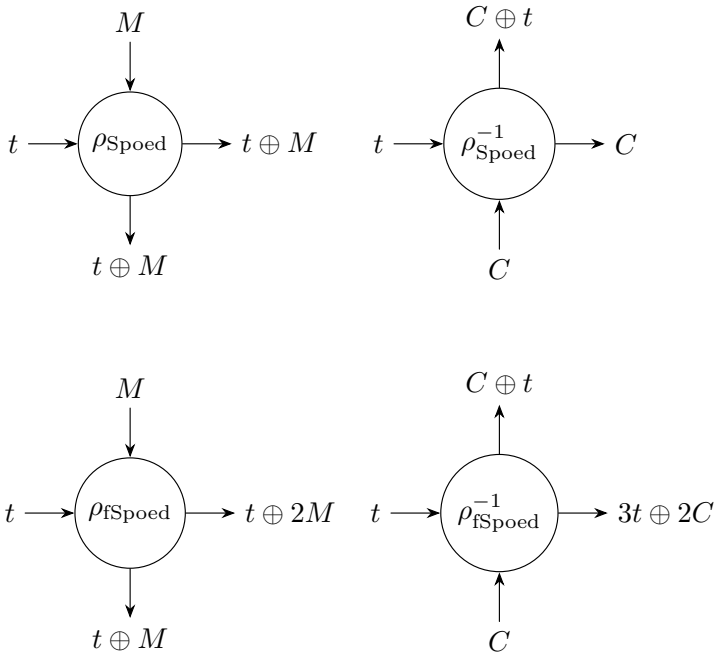


Fig. 5. Feedback functions of Spoed and fSpoed.

Concretely, while Spoed updates its internal state using a linear combination of the previous state and the left half of the current GPAD block, this update does not sufficiently link adjacent blocks during verification, allowing distinct inputs to induce identical internal states in later rounds. As shown in Section 3.2, this structural property enables an adversary to construct fresh decryption queries that pass verification with probability 1.

In contrast, fSpoed introduces a slight modification to the feedback term in the state update. This change ensures that any difference in earlier blocks is necessarily propagated to all subsequent internal states, thereby preventing the coincidence of inputs to the final compression-function evaluation during verification. As a result, the forgery and plaintext-recovery attacks described in the previous sections no longer apply.

Feedback Functions in Spoed *and* fSpoed. Feedback functions offer a structural viewpoint for analyzing feedback-based authenticated encryption schemes and have been adopted in several modern AE designs, including COFB [CIMN17], Beetle [CDNY18] and COLM [ABD+24]. Conceptually, a feedback function specifies two tightly coupled mechanisms: the evolution of the internal chaining state across processing steps and the reversible blockwise transformation between plaintext and ciphertext. Given a current state t and a plaintext block M, the forward mapping $\rho(t, M) = (s, C)$ determines both the next state s and

the emitted ciphertext block C, while the inverse mapping $\rho^{-1}(t, C) = (s, M)$ guarantees consistent plaintext recovery under the same state evolution. In this way, the pair (ρ, ρ^{-1}) fully characterizes the feedback dynamics of the encryption mode.

For Spoed and its patched variant fSpoed, the feedback functions admit simple closed-form expressions:

$$\rho_{\mathrm{Spoed}}(t, M) = \big(t \oplus M,\ t \oplus M\big),$$
$$\rho_{\mathrm{fSpoed}}(t, M) = \big(t \oplus M,\ t \oplus 2M\big).$$

Their corresponding inverse mappings are

$$\rho^{-1}_{\mathrm{Spoed}}(t, C) = \big(C,\ C \oplus t\big),$$
$$\rho^{-1}_{\mathrm{fSpoed}}(t, C) = \big(3t \oplus 2C,\ C \oplus t\big).$$

These expressions expose the essential structural distinction between the two schemes, see Fig. 5. In Spoed, the next state coincides with the ciphertext block, creating a degeneracy in the feedback dynamics that allows different transcripts to induce identical internal inputs to the underlying compression function. This structural alignment is precisely what enables the attacks developed later in the paper. In contrast, fSpoed separates the state evolution from ciphertext generation through the linear coefficient 2, thereby preventing such alignment and restoring a non-degenerate feedback structure while preserving efficiency and design simplicity.

In the following, we formally explain in detail why the original security proof of Spoed fails, and why the modified construction fSpoed restores the claimed confidentiality and integrity guarantees under the nonce-respecting setting.

5 Provable Security of fSpoed

In this section, we demonstrate why the original proof of Spoed fails and why our modification fSpoed is effective.

H-Coefficient Technique. Let $\Delta_{\mathcal{A}}(\widetilde{F}; \widetilde{R})$ denote the distinguishing advantage of an adversary $\mathcal{A}$ between a "real" oracle $\widetilde{F}$ and an "ideal" oracle $\widetilde{R}$. Let $X_{\widetilde{F}}$ and $X_{\widetilde{R}}$ be the distributions of transcripts induced by the interaction of $\mathcal{A}$ with $\widetilde{F}$ and $\widetilde{R}$, respectively, and let $\mathcal{V}$ denote the set of all attainable transcripts.

In the H-coefficient technique, the transcript space $\mathcal{V}$ is partitioned into a set of good transcripts $\mathcal{V}_{\mathrm{good}}$ and a set of bad transcripts $\mathcal{V}_{\mathrm{bad}}$, where $\mathcal{V}_{\mathrm{good}} \cup \mathcal{V}_{\mathrm{bad}} = \mathcal{V}$. Suppose that there exists $\varepsilon \geq 0$ such that for every $v \in \mathcal{V}_{\mathrm{good}}$,

$$\frac{\Pr[X_{\widetilde{F}} = v]}{\Pr[X_{\widetilde{R}} = v]} \geq 1 - \varepsilon.$$

Then, for any adversary $\mathcal{A}$, it holds that

$$\Delta_{\mathcal{A}}(\widetilde{F}; \widetilde{R}) \leq \varepsilon + \Pr\big[X_{\widetilde{R}} \in \mathcal{V}_{\mathrm{bad}}\big].$$

Ashur and Mennink proved the authenticated-encryption security of Spoed using the H-coefficient technique [Pat09], a proof methodology that has become widely adopted in symmetric-cryptographic security analyses since [CS14]. At a high level, their argument follows a two-stage reduction.

In the first stage, every occurrence of $F_{i,L}(X,Y) \coloneqq F_K(X \oplus 2^i L, Y)$ is replaced by its idealized counterpart, namely a tweakable random function. Using the H-coefficient technique, they bound the distinguishing advantage of any adversary making at most q queries over a total of at most σ processed blocks when distinguishing the real family $F_{i,L}$ from an ideal oracle $\widetilde{R}$.

In the second stage, they analyze the privacy and integrity of an *idealized* version of Spoed in which $F_{i,L}$ is instantiated by the ideal oracle $\widetilde{R}$. This modular proof structure separates primitive security from mode-level security, thereby simplifying both verification and readability of the overall argument. The first stage yields the following lemma.

Lemma 2 ([AM16]). *Let $F : \{0,1\}^k \times \{0,1\}^{2n} \to \{0,1\}^n$ be a keyed compression function. Let $\mathcal{T} = \{1,2,\ldots,2^{n/2}\} \times \{0,1\}^* \times \{0,1\}^n$, and define $\widetilde{F} : \{0,1\}^k \times \mathcal{T} \times \{0,1\}^{2n} \to \{0,1\}^n$ by*

$$\widetilde{F}\big(K,(\alpha,\beta,N),S\big) = F\big(K,\big(2^\alpha 3^\beta \cdot F_K(N\|0) \,\|\, 0^n\big) \oplus S\big)\,.$$

Then

$$\mathbf{Adv}^{\widetilde{\mathrm{prf}}}_{\widetilde{F}}(q,t) \le \frac{1.5q^2}{2^n} + \mathbf{Adv}^{\mathrm{prf}}_{F}(2q,t'),$$

where $t' \approx t$.

*Applicability of Lemma 2 to f*Spoed. The modification introduced in fSpoed preserves the structural conditions required for Lemma 2. In particular, the keyed compression function evaluated within fSpoed can still be expressed in the form

$$\widetilde{F}_K(\alpha,\beta,N;S) = F_K\big(S \oplus \Delta_{\alpha,\beta}(N)\big),$$

where $\Delta_{\alpha,\beta}(N)$ is a publicly computable offset determined by the nonce N together with fixed public constants.

The introduction of an additional public constantnamely the multiplicative factor 2-only alters the concrete definition of the offset mapping $\Delta_{\alpha,\beta}$, while preserving its injectivity and the cardinality of the associated tweak space. Consequently, the H-coefficientbased argument underlying Lemma 2 remains valid for fSpoed without modification, and the same tPRF replacement bound continues to hold.

Their Case Study of Ideal Spoed. We revisit the case analysis used in the proof of ideal Spoed and identify a critical mismatch between the considered collision event and the actual verification behavior. Suppose the adversary submits a verification query (N,A,C,T) after having issued encryption queries (N^j,A^j,M^j) for $j = 1,\ldots,q_{\mathcal{E}}$, with corresponding ciphertexttag pairs (C^j,T^j). The proof

defines the event col to occur if there exists an encryption query j with $N^j = N$, $\ell^j = \ell$, and an index $i \in \{1, \ldots, \ell\}$ such that

$$t^j_{i-1} \oplus Z^{0j}_i \| Z^{1j}_i \neq t_{i-1} \oplus Z^0_i \| Z^1_i \ \wedge \ t^j_i = t_i. \tag{1}$$

Examining the decryption process reveals that $t_{i-1} \oplus Z^0_i = C_{i-1}$, so that Equation (1) reduces to

$$C^j_{i-1} \| A^j_i \neq C_{i-1} \| A_i \ \wedge \ t^j_i = t_i.$$

This formulation exposes the fundamental gap in the proof: the ciphertext blocks are not cryptographically chained across subsequent positions. Consequently, a modification confined to the i-th block affects only the next decryption step and does not propagate to later internal states. An adversary can therefore construct a ciphertext whose $(i-1)$-st block matches the target transcript while introducing differences strictly before index $i-1$, yielding a trivial state collision from position i onward.

The corrected collision condition should instead capture equality of the *final* internal state. Formally, a rectified col event occurs if there exist an index $i \in \{1, \ldots, \ell\}$ and an encryption query j with $N^j = N$ and $\ell^j = \ell$ such that

$$Z^{0j}_i \neq Z^0_i \implies t^j_{\ell+1} = t_{\ell+1}.$$

This condition reflects the true verification behavior and explains why the original H-coefficientbased argument fails for Spoed.

Motivation of the col *Event.* In the second stage of the proof, the goal is to bound the advantage of an adversary distinguishing ideal Spoed from an ideal authenticated-encryption scheme. Because the adversary is nonce-respecting, encryption queries necessarily use distinct nonces. When the underlying primitive is replaced by a tweakable random function, the randomness of encryption becomes indistinguishable from the ideal AE behavior, and the security analysis therefore reduces to bounding the *integrity of ciphertexts* under verification queries.

To capture this requirement, the proof introduces the collision event col, which is intended to characterize when a verification query could be consistent with a prior encryption transcript. The rectified version of col directly reflects the verification procedure by requiring equality of the final internal state. However, as shown by our analysis, there exists an adversary for which the rectified col event occurs with probability 1. Such an adversary can in fact be derived from our simple universal forgery attack, demonstrating that the integrity bound in the original argument collapses.

This observation raises the natural question of why the modified scheme fSpoed avoids this failure and how its security proof can be re-established.

Why our patch works. The effectiveness of the patch in fSpoed stems from the modified linear relation in the feedback computation. Specifically, the term $t_{i-1} \oplus Z^0_i$ in Spoed is replaced by $t_{i-1} \oplus 2 \cdot Z^0_i$, which alters the algebraic structure

of the state update during verification. Substituting the verification relation yields

$$t_{i-1} \oplus 2\big(Z_i^0 \oplus t_{i-1}\big) = 3t_{i-1} \oplus 2Z_i^0.$$

Unlike the original expression in Spoed, where the dependence on t_{i-1} cancels and enables trivial state alignment, the modified affine combination in fSpoed preserves a non-degenerate dependence on both the prior state and the message-derived component. As a result, any difference introduced in the ciphertext necessarily propagates through subsequent evaluations of the underlying ideal primitive $\widetilde{F}$, preventing the collision structure that enables the forgery attack.

This structural separation restores the integrity argument and leads to the following lemma.

Integrity in the Ideal World. We analyze the integrity of fSpoed in the ideal world, where each instance of the keyed compression function $\widetilde{F}_K$ is replaced by an independent tweakable random function. In this setting, a decryption query (N, A, C, T) is accepted if and only if the final internal value t_ℓ satisfies

$$T = \mathrm{left}_\tau(t_\ell).$$

Thus, for any fresh verification query, a successful forgery can occur only in one of two cases: either the corresponding tweak has never been queried beforeyielding success probability $2^{-\tau}$ due to tag truncationor the internal state collides with that of a previous encryption query, corresponding to the event colD defined below.

Lemma 3. *For any nonce-respecting adversary making at most $q_\mathcal{E}$ encryption queries and $q_\mathcal{D}$ verification queries, processing at most σ blocks in total and at most ℓ blocks per query, it holds that*

$$\mathbf{Adv}^{\mathrm{int}}_{\mathsf{idealfSpoed}}(\mathsf{nr}, q_\mathcal{E}, q_\mathcal{D}, \ell, \sigma) \le \frac{\ell\, q_\mathcal{D}}{2^n} + \frac{q_\mathcal{D}}{2^\tau}.$$

Proof. Let (N^j, A^j, M^j) denote the j-th encryption query of $\mathcal{A}$ for $j = 1, \ldots, q_\mathcal{E}$, with corresponding ciphertexttag pairs (C^j, T^j). Write

$$(Z_1^j, \ldots, Z_{\ell^j}^j) = \mathrm{GPAD}_{n,\tau}(A^j, M^j),$$

and denote the internal inputs and outputs of the independent tweakable random functions by (s_i^j, t_i^j) for $i = 1, \ldots, \ell^j$.

By the standard single-verification reduction of BellareGoldreichMicali [BGM04], it suffices to analyze a single verification query (N, A, C, T) and multiply the resulting bound by $q_\mathcal{D}$. Let its block length be ℓ, and denote the internal states during decryption by (s_i, t_i) with

$$(Z_1, \ldots, Z_\ell) = \mathrm{GPAD}_{n,\tau}(A, M).$$

A forgery attempt succeeds if and only if

$$T = \mathrm{left}_\tau(t_\ell).$$

Define the collision event colD as the existence of an encryption query j with $N^j = N$ and $\ell^j = \ell$, together with an index $i \in \{1, \ldots, \ell\}$ such that $Z_i^j \neq Z_i$ while the final states coincide, i.e., $t_{\ell+1}^j = t_{\ell+1}$. Because the adversary is nonce-respecting, the evaluations of the tweakable random functions from position i onward are independent, and hence the probability that the internal states collide is bounded by a union bound:

$$\Pr[\mathsf{colD}] \leq \sum_{k=i}^{\ell} \frac{1}{2^n} \leq \frac{\ell}{2^n}.$$

We now distinguish two exhaustive cases.

1. If the verification query uses a fresh tweaki.e., $N \notin \{N^1, \ldots, N^{q_\mathcal{E}}\}$ or $\ell \neq \ell^j$ for all $j$then the corresponding tweakable random function output has never been queried before, and the success probability is at most $2^{-\tau}$ due to tag truncation.
2. Otherwise, any successful forgery implies the event colD, whose probability is bounded by $\ell/2^n$.

Combining the two cases and multiplying by $q_\mathcal{D}$ yields the claimed bound.

Combining the Bounds. We derive the overall security bounds of fSpoed via a standard hybrid argument. By Lemma 2, the real-world execution of fSpoed can be replaced by an idealized construction in which the underlying keyed compression function is instantiated by independent tweakable random functions, incurring a replacement cost of at most

$$\frac{1.5\,\sigma^2}{2^n} \;+\; \mathbf{Adv}_F^{\mathrm{prf}}(2\sigma, t').$$

In this ideal world, confidentiality follows immediately from the nonce-respecting assumption, since each ciphertext block is masked by an independent uniform value and the encryption oracle is therefore indistinguishable from the ideal oracle \$. Integrity in the ideal world is bounded by Lemma 3.

Combining the replacement bound with the ideal-world confidentiality and integrity bounds yields the following theorem.

Theorem 2. *For any nonce-respecting adversary, it holds that*

$$\mathbf{Adv}_{\mathrm{fSpoed}}^{\mathrm{conf}}(\mathsf{nr}, q, \ell, \sigma, t) \leq \frac{1.5\,\sigma^2}{2^n} + \mathbf{Adv}_F^{\mathrm{prf}}(2\sigma, t'),$$

$$\mathbf{Adv}_{\mathrm{fSpoed}}^{\mathrm{int}}(\mathsf{nr}, q_\mathcal{E}, q_\mathcal{D}, \ell, \sigma, t) \leq \frac{1.5\,\sigma^2}{2^n} + \frac{\ell\, q_\mathcal{D}}{2^n} + \frac{q_\mathcal{D}}{2^\tau} + \mathbf{Adv}_F^{\mathrm{prf}}(2\sigma, t'),$$

where $t' \approx t$.

6 Conclusion

In this paper, we identified a fundamental structural weakness in Spoed and showed that it breaks both integrity and confidentiality. We demonstrated that this weakness enables a probability-1 universal forgery attack and efficient (almost) plaintext recovery with minimal query complexity, and that the failure is inherent to the feedback structure of the scheme rather than to the choice of the underlying compression function.

To remedy this issue, we proposed a minimally modified variant, denoted fSpoed. The modification preserves the overall design philosophy and efficiency of Spoed, while eliminating the structural flaw exploited by our attacks. Using a refined security analysis, we showed that fSpoed achieves the originally claimed confidentiality and integrity guarantees in the nonce-respecting setting under standard assumptions. Our results highlight the importance of carefully designed feedback mechanisms in compression-functionbased authenticated encryption schemes.

References

ABD+24. Elena Andreeva, Andrey Bogdanov, Nilanjan Datta, Atul Luykx, Bart Mennink, Mridul Nandi, Elmar Tischhauser, and Kan Yasuda. The COLM Authenticated Encryption Scheme. *Journal of Cryptology*, 37(2):15, March 2024. 11

AM16. Tomer Ashur and Bart Mennink. Damaging, Simplifying, and Salvaging p-OMD. In Matt Bishop and Anderson C A Nascimento, editors, *Information Security*, pages 73–92, Cham, 2016. Springer International Publishing. 1, 6, 8, 13, 19

BGM04. Mihir Bellare, Oded Goldreich, and Anton Mityagin. The Power of Verification Queries in Message Authentication and Authenticated Encryption, 2004. 15

CDNY18. Avik Chakraborti, Nilanjan Datta, Mridul Nandi, and Kan Yasuda. Beetle Family of Lightweight and Secure Authenticated Encryption Ciphers. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 218–241, May 2018. 11

CIMN17. Avik Chakraborti, Tetsu Iwata, Kazuhiko Minematsu, and Mridul Nandi. Blockcipher-Based Authenticated Encryption: How Small Can We Go? In Wieland Fischer and Naofumi Homma, editors, *Cryptographic Hardware and Embedded Systems – CHES 2017*, Lecture Notes in Computer Science, pages 277–298, Cham, 2017. Springer International Publishing. 11

CMN+14a. Simon Cogliani, D. S. Maimut, David Naccache, Rodrigo Portella do Canto, Reza Reyhanitabar, Serge Vaudenay, and D. Vizźar. Offset merkle-damgård (omd) version 1.0 a caesar proposal. *Proposal in CAESAR competition (March 2014)*, 2014. 1

CMN+14b. Simon Cogliani, Diana-Ştefania Maimuţ, David Naccache, Rodrigo Portella do Canto, Reza Reyhanitabar, Serge Vaudenay, and Damian Vizár. OMD: A Compression Function Mode of Operation for Authenticated Encryption. In Antoine Joux and Amr Youssef, editors, *Selected Areas in Cryptography – SAC 2014*, Lecture Notes in Computer Science, pages 112–128, Cham, 2014. Springer International Publishing. 1

CMN+15. Simon Cogliani, Diana Maimut, David Naccache, Rodrigo Portella, Reza Reyhanitabar, and Damian Vizár. Offset Merkle-Damgård (OMD) version 2.0 A CAESAR Proposal. August 2015. 1

CS14. Shan Chen and John Steinberger. Tight Security Bounds for Key-Alternating Ciphers. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, Lecture Notes in Computer Science, pages 327–350, Berlin, Heidelberg, 2014. Springer. 13

IIMP19. Akiko Inoue, Tetsu Iwata, Kazuhiko Minematsu, and Bertram Poettering. Cryptanalysis of OCB2: Attacks on Authenticity and Confidentiality. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019*, Lecture Notes in Computer Science, pages 3–31, Cham, 2019. Springer International Publishing. 2

IIMP20. Akiko Inoue, Tetsu Iwata, Kazuhiko Minematsu, and Bertram Poettering. Cryptanalysis of OCB2: Attacks on Authenticity and Confidentiality. *Journal of Cryptology*, 33(4):1871–1913, October 2020. 2

Pat09. Jacques Patarin. The "Coefficients H" Technique. In Roberto Maria Avanzi, Liam Keliher, and Francesco Sica, editors, *Selected Areas in Cryptography*, Lecture Notes in Computer Science, pages 328–345, Berlin, Heidelberg, 2009. Springer. 13

RVV14. Reza Reyhanitabar, Serge Vaudenay, and Damian Vizár. Misuse-Resistant Variants of the OMD Authenticated Encryption Mode. In Sherman S. M. Chow, Joseph K. Liu, Lucas C. K. Hui, and Siu Ming Yiu, editors, *Provable Security*, Lecture Notes in Computer Science, pages 55–70, Cham, 2014. Springer International Publishing. 1

RVV15. Reza Reyhanitabar, Serge Vaudenay, and Damian Vizár. Boosting OMD for Almost Free Authentication of Associated Data. In Gregor Leander, editor, *Fast Software Encryption*, Lecture Notes in Computer Science, pages 411–427, Berlin, Heidelberg, 2015. Springer. 1

A Specification of Spoed

We first clarify a minor inconsistency in the published specification of Spoed. In the original paper, Line 12 of the decryption algorithm $\mathcal{D}$ is given as

$$t_\ell = F_K\left(t_{\ell-1} \oplus 2^\ell 3L \oplus Z_{\ell-1} \,\|\, Z_\ell^1\right).$$

This expression appears to be a typographical inconsistency. For correctness, the decryption computation must match the corresponding encryption step (Line 9 in Fig. 6), and the term $Z_{\ell-1}$ should instead be the last plaintext block $M_{\ell-1}$. Accordingly, the corrected expression is

$$t_\ell = F_K\left(t_{\ell-1} \oplus 2^\ell 3L \oplus M_{\ell-1} \,\|\, Z_\ell^1\right).$$

This correction follows from the basic correctness requirement of authenticated encryption: any tuple produced by the encryption procedure must decrypt to the original message. When $m \geq a$, the GPAD encoding yields $Z_\ell^0 = M_m$ and $\ell = \max\left(m, \frac{a+m}{2}\right) + 1 = m + 1$, so that $\ell - 1 = m$. Hence the decryption input must contain $M_{\ell-1}$.

Spoed encryption $\mathcal{E}$

Input : (K, N, A, M)
Output: (C, T)

$(Z_1, \ldots, Z_l) = \mathrm{GPAD}_{n,\tau}(A, M)$
$m = \lceil |M| / n \rceil$
$L = F_K(N \| 0)$
$t_0 = 0^n$
for $i = 1$ **to** $\ell - 1$ **do**
  $t_i = F_K\left(t_{i-1} \oplus 2^i L \oplus Z_i^0 \| Z_i^1\right)$
  $C_i = t_i \oplus Z_{i+1}^0$
end
$t_\ell = F_K\left(t_{\ell-1} \oplus 2^\ell 3L \oplus Z_\ell^0 \| Z_\ell^1\right)$
$C = \mathrm{left}_{|M|}(C_1 \| \cdots \| C_{\ell-1})$
$T = \mathrm{left}_\tau(t_\ell)$
return (C, T)

Spoed decryption $\mathcal{D}$

Input : (K, N, A, C, T)
Output: M or $\perp$

$(Z_1, \ldots, Z_l) = \mathrm{GPAD}_{n,\tau}(A, C)$
$m = \lceil |C| / n \rceil$
$\rho = |C| \bmod n$
$L = F_K(N \| 0)$
$t_0 = 0^n, M_0 = Z_1^0$
for $i = 1$ **to** $\ell - 1$ **do**
  $t_i = F_K\left(t_{i-1} \oplus 2^i L \oplus M_{i-1} \| Z_i^1\right)$
  if $i < m$ **then** $M_i = t_i \oplus Z_{i+1}^0$
  if $i = m$ **then** $M_i = \mathrm{left}_\rho(t_i) \oplus Z_{i+1}^0$
  if $i > m$ **then** $M_i = Z_{i+1}^0$
end
$t_\ell = F_K\left(t_{\ell-1} \oplus 2^\ell 3L \oplus Z_\ell^0 \| Z_\ell^1\right)$
$M = \mathrm{left}_{|C|}(M_1 \| \cdots \| M_{\ell-1})$
$T' = \mathrm{left}_\tau(t_\ell)$
return $T = T' ? M : \perp$

Fig. 6. The original specification of Spoed, adapted from [AM16].

To illustrate the consequence of the inconsistent specification, consider the encryption and decryption of a two-block message $(N, A_1 \| A_2, M_1 \| M_2)$, as depicted in Fig. 7. The intermediate value highlighted in red should coincide in both procedures. However, under the original expression, the encryption side produces $t_2 \oplus M_2$, while the decryption side yields $t_2 \oplus C_2 = t_2 \oplus M_2 \oplus t_2 = M_2$. These values agree only when $t_2 = 0^n$, which contradicts the correctness requirement of authenticated encryption except with negligible probability.

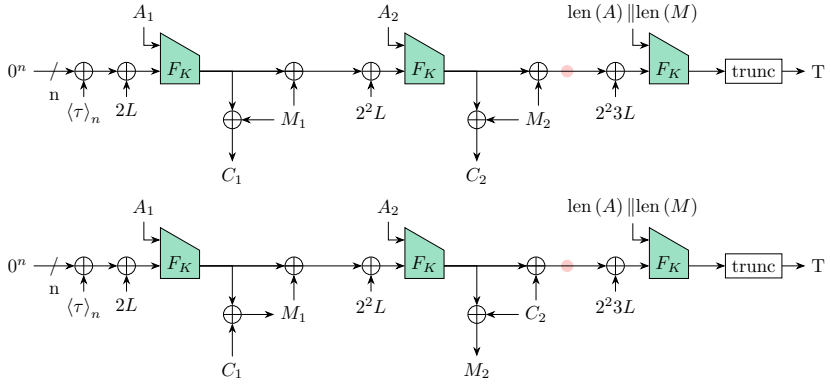


Fig. 7. Illustration of the inconsistent specification of Spoed for a two-block message. The highlighted intermediate value must coincide in encryption and decryption for correctness, but the original specification produces distinct values with overwhelming probability.